VitalFlow V1 Permission Matrix

Status: Design proposal. The current runtime (packages/auth/src/rbac.ts) uses a coarser first-pass permission set; this document defines the module-level V1 model and the migration path from today.

---

1. Scope

17 modules × 8 roles × 9 actions.

Modules

#	Module	Primary tables	DB exists?
1	`patient_records`	`patients`, `patient_contacts`, `patient_coverages`	✅
2	`appointments`	`appointments`	✅
3	`encounters`	`encounters`	✅
4	`notes`	`encounter_notes`, `signatures`	✅
5	`clinical_lists` (problems/allergies/meds)	`problems`, `allergies`, `medications`, `immunizations`, `vitals`	✅
6	`intake_forms`	*(TBD)*	❌
7	`check_in`	*(uses `appointments.status`)*	⚠️ partial
8	`billing_dashboard`	`invoices`, `invoice_lines`, `payments`, `charges`	✅
9	`claims`	`claims`, `claim_lines`	✅
10	`denials`	`claim_status_history` (status=denied)	✅
11	`staff_records`	`profiles`, `tenant_members`	✅
12	`credentials`	*(TBD — license/DEA/NPI tracking)*	❌
13	`tasks`	`tasks`, `task_comments`	✅
14	`inventory`	`inventory_items`, `inventory_transactions`	✅
15	`admin_settings`	`tenants`, `feature_flags`	✅
16	`audit_logs`	`audit.*`	✅
17	`entitlements`	`entitlements`, `feature_flag_overrides`	✅

Roles: `super_admin` (platform), `practice_owner`, `office_admin`, `physician`, `nurse_ma`, `scheduler`, `biller`, `patient`.

Actions: `view`, `create`, `update`, `delete`, `sign`, `export`, `assign`, `approve`, `manage`.

Not every action applies to every module — see §4.

---

2. Design principles

1. **Tenant isolation is not a permission** — it's an RLS predicate. Every query adds `tenant_id = current_tenant_id()`. A role never grants cross-tenant access.
2. **Three enforcement layers, in order:**
   - **Postgres RLS** — authoritative. Runs inside the DB, can't be bypassed by a forgotten check in the app.
   - **SQL** `public.has_permission(role[], permission)` — used in RLS predicates and RPCs.
   - **TypeScript guards** — fast UI feedback + defense-in-depth for actions that can't be expressed in RLS (e.g., export pipelines).
3. **UI hides, backend enforces.** A disabled button is a UX courtesy; a 403 on the server is the real check.
4. **Patient users are fully separated.** `user_kind='patient'` cannot be assigned any staff permission. Patient permissions are `self:*` only, scoped via `patient_portal_links`.
5. **Impersonation strips sensitive perms.** See §7.
6. **Deletion is rare.** HIPAA retention rules mean most "deletes" are soft (`deleted_at`). Hard delete is `manage`-gated and audit-logged.

---

3. Permission key convention

Format: `<module>:<action>`

```
patient_records:view
patient_records:create
patient_records:update
patient_records:delete
patient_records:export
encounters:sign
notes:amend
claims:submit
claims:approve
tasks:assign
staff_records:manage
audit_logs:export
```

Special namespaces kept:

- `self:*` — patient self-service only
- `platform:*` — super_admin only, cross-tenant
- `ai:*` — cross-cutting (`ai:invoke`, `ai:train`)

See `packages/auth/src/permissions-v2.ts` for the canonical enum.

---

4. Permission matrix

Legend: ✅ = granted, ⬜ = not granted, 🟠 = conditional (see footnotes), ★ = requires two-person approval or impersonation block.

4.1 Clinical surface

Module / Action	super_admin	practice_owner	office_admin	physician	nurse_ma	scheduler	biller	patient

`patient_records:view`	✅	✅	✅	✅	✅	🟡 demog[1]	✅	🟡 self[2]
`patient_records:create`	✅	✅	✅	✅	✅	✅	⬜	⬜
`patient_records:update`	✅	✅	✅	✅	✅	🟡 demog[1]	⬜	🟡 self[2]
`patient_records:delete`	✅	⬜	⬜	⬜	⬜	⬜	⬜	⬜
`patient_records:export`	✅	✅★	✅★	⬜	⬜	⬜	⬜	🟡 self[2]
`encounters:view`	✅	✅	✅	✅	✅	⬜	✅	🟡 self[2]
`encounters:create`	✅	✅	⬜	✅	✅	⬜	⬜	⬜
`encounters:update`	✅	✅	⬜	✅	🟡 pre-sign[3]	⬜	⬜	⬜
`encounters:sign`	⬜	✅	⬜	✅	⬜	⬜	⬜	⬜
`encounters:amend`	⬜	✅★	⬜	✅★	⬜	⬜	⬜	⬜
`notes:view`	✅	✅	🟡 non-psych[4]	✅	✅	⬜	🟡 non-psych[4]	🟡 self[2]
`notes:create`	✅	✅	⬜	✅	✅	⬜	⬜	⬜
`notes:update`	✅	✅	⬜	✅	🟡 pre-sign[3]	⬜	⬜	⬜
`notes:sign`	⬜	✅	⬜	✅	⬜	⬜	⬜	⬜
`notes:amend`	⬜	✅★	⬜	✅★	⬜	⬜	⬜	⬜
`notes:export`	✅	✅★	✅★	⬜	⬜	⬜	⬜	🟡 self[2]
`clinical_lists:view`	✅	✅	⬜	✅	✅	⬜	⬜	🟡 self[2]
`clinical_lists:update`	✅	✅	⬜	✅	✅	⬜	⬜	⬜
`clinical_lists:sign`	⬜	✅	⬜	✅	⬜	⬜	⬜	⬜

[1] **demographics only** — name, DOB, phone, address, insurance header. No clinical data. [2] **self** — patient can view/update only their own record via `patient_portal_links`. [3] **pre-sign** — can edit until the physician signs; after sign, requires amend. [4] **non-psych** — excludes notes flagged as psychotherapy (45 CFR 164.501). ★ **two-person / audit-heavy** — export and amend actions write an `audit.audit_log` entry and (for bulk export) require a second approver.

4.2 Front-office surface

Module / Action	super_admin	practice_owner	office_admin	physician	nurse_ma	scheduler	biller	patient
`appointments:view`	✅	✅	✅	✅	✅	✅	⬜	🟡 self[2]
`appointments:create`	✅	✅	✅	⬜	⬜	✅	⬜	🟡 self[2]
`appointments:update`	✅	✅	✅	⬜	⬜	✅	⬜	🟡 self[2]
`appointments:delete`	✅	✅	✅	⬜	⬜	✅	⬜	⬜

`appointments:assign`	✅	✅	✅	⬜	⬜	✅	⬜	⬜
`intake_forms:view`	✅	✅	✅	✅	✅	✅	⬜	🟡 self[2]
`intake_forms:create`	✅	✅	✅	⬜	⬜	✅	⬜	⬜
`intake_forms:update`	✅	✅	✅	⬜	⬜	✅	⬜	🟡 self[2]
`check_in:view`	✅	✅	✅	✅	✅	✅	⬜	🟡 self[2]
`check_in:update`	✅	✅	✅	⬜	✅	✅	⬜	🟡 self[2]

4.3 Revenue cycle surface

Module / Action	super_admin	practice_owner	office_admin	physician	nurse_ma	scheduler	biller	patient
`billing_dashboard:view`	✅	✅	✅	⬜	⬜	⬜	✅	⬜
`claims:view`	✅	✅	✅	⬜	⬜	⬜	✅	⬜
`claims:create`	✅	✅	✅	⬜	⬜	⬜	✅	⬜
`claims:update`	✅	✅	✅	⬜	⬜	⬜	✅	⬜
`claims:submit`	✅	✅	✅	⬜	⬜	⬜	✅	⬜
`claims:approve`	✅	✅★	⬜	⬜	⬜	⬜	⬜	⬜
`claims:export`	✅	✅	✅	⬜	⬜	⬜	✅	⬜
`denials:view`	✅	✅	✅	⬜	⬜	⬜	✅	⬜
`denials:update`	✅	✅	✅	⬜	⬜	⬜	✅	⬜
`denials:approve`	✅	✅★	⬜	⬜	⬜	⬜	⬜	⬜
`billing:adjust`	✅	✅★	⬜	⬜	⬜	⬜	✅🟡[5]	⬜
`billing:write_off`	✅	✅★	⬜	⬜	⬜	⬜	🟡 <$500[5]	⬜

[5] **threshold-gated** — biller can write off up to a tenant-configured limit without approval; above that, owner sign-off required. Enforced server-side via `billing:approve_writeoff` check in the RPC.

4.4 Administrative surface

Module / Action	super_admin	practice_owner	office_admin	physician	nurse_ma	scheduler	biller	patient
`staff_records:view`	✅	✅	✅	⬜	⬜	⬜	⬜	⬜
`staff_records:create`	✅	✅	✅	⬜	⬜	⬜	⬜	⬜
`staff_records:update`	✅	✅	✅	⬜	⬜	⬜	⬜	⬜
`staff_records:manage`	✅	✅	🟡 not owner[6]	⬜	⬜	⬜	⬜	⬜
`credentials:view`	✅	✅	✅	🟡 self[7]	🟡 self[7]	⬜	⬜	⬜
`credentials:update`	✅	✅	✅	🟡 self[7]	🟡 self[7]	⬜	⬜	⬜
`credentials:approve`	✅	✅★	⬜	⬜	⬜	⬜	⬜	⬜

`tasks:view`	✅	✅	✅	✅	✅	✅	✅	⬜
`tasks:create`	✅	✅	✅	✅	✅	✅	✅	⬜
`tasks:assign`	✅	✅	✅	✅	🟡 same-team	✅	✅	⬜
`tasks:update`	✅	✅	✅	🟡 assigned	🟡 assigned	🟡 assigned	🟡 assigned	⬜
`inventory:view`	✅	✅	✅	⬜	✅	⬜	⬜	⬜
`inventory:update`	✅	✅	✅	⬜	✅	⬜	⬜	⬜
`inventory:manage`	✅	✅	✅	⬜	⬜	⬜	⬜	⬜

[6] **not owner** — office_admin can remove/edit anyone *except* `practice_owner`. Only another owner can demote an owner. [7] **self** — clinicians manage their own DEA/NPI/license records but not peers'.

4.5 Platform & audit surface

Module / Action	super_admin	practice_owner	office_admin	physician	nurse_ma	scheduler	biller	patient
`admin_settings:view`	✅	✅	✅	⬜	⬜	⬜	⬜	⬜
`admin_settings:update`	✅	✅	🟡 non-billing[8]	⬜	⬜	⬜	⬜	⬜
`admin_settings:manage`	✅	✅	⬜	⬜	⬜	⬜	⬜	⬜
`audit_logs:view`	✅	✅	✅	⬜	⬜	⬜	⬜	⬜
`audit_logs:export`	✅	✅★	⬜	⬜	⬜	⬜	⬜	⬜
`entitlements:view`	✅	✅	✅	⬜	⬜	⬜	⬜	⬜
`entitlements:manage`	✅	⬜	⬜	⬜	⬜	⬜	⬜	⬜

[8] **non-billing** — office_admin can change branding, hours, integrations, but not pricing/plan/BAA flags (those are `admin_settings:manage`).

---

5. Route protection rules

Every Next.js route maps to a required permission set and surface group:

```
// Convention: colocate a `route.meta.ts` sibling to page.tsx
export const meta = {
  surface: "provider",                  // or "admin" | "patient"
  requires: ["encounters:view"],        // at least one
  requiresAll: [],                      // must have all if set
  impersonationAllowed: true,
};
```

Enforcement order (top of every page/action):

1. `requireSession()` → 401 if no cookie or Supabase session.

2. `requireSurface(ctx, meta.surface)` → 403 if wrong `user_kind` or no surface anchor permission.
3. `requireAnyPermission(ctx, meta.requires)` → 403.
4. `requireAllPermissions(ctx, meta.requiresAll)` → 403.
5. If `!meta.impersonationAllowed && isImpersonating(ctx)` → 403.

Middleware responsibility (`apps/web/src/middleware.ts`):

- Refresh Supabase session cookie (already done).
- Attach `x-vf-tenant-id` header for downstream.
- Do **not** perform permission checks — that's per-route.

404 vs 403 policy: unknown route → 404. Known route, no permission → 403. Don't leak existence of admin-only pages via distinct errors — `/admin/billing` returns 404 for a physician (permission-hide), not 403.

---

6. UI visibility rules

1. **Nav filtering** (apps/web/src/nav/*.ts) — every item has a `requires: Permission[]` . The shell filters at render. This is the *only* place UI visibility logic lives.
2. **Action buttons** — each button declares `requires` . Hidden (not disabled) when missing. Example: a "Sign note" button is invisible to `nurse_ma` .
3. **Read-only fallback** — when a user has `view` but not `update` , render the same form read-only. Do not hide the data.
4. **Never rely on UI alone** — every action that changes data re-checks permission server-side.
5. **Empty-state messaging** — when RLS filters out all rows (e.g. scheduler viewing notes), show "You don't have access to this information" rather than "No records found" — users can tell the difference and it's an integrity signal.

---

7. Backend enforcement rules

1. **Postgres RLS** is the backstop. Every table has a policy. Example:

```
create policy "notes_read" on public.encounter_notes
  for select using (
    public.is_member_of(tenant_id)
    and public.has_permission(public.current_roles(tenant_id), 'notes:view')
  );
```

2. **Row-shape filtering** — `patient_records:view` with scheduler role returns only demographic columns. Enforce via a **view** (`public.patients_demographics_v`) that RLS-protects the full table.
3. **Column-level** — for psychotherapy notes, use a `psych` boolean column + RLS predicate excluding it from `notes:view` unless caller has `clinical:sign` (proxy for "is a clinician").
4. **Server actions** call `requireAllPermissions(ctx, [...])` at the top. Never pass `ctx` from the client.
5. **RPCs for privileged writes** — `claims:approve` , `billing:adjust` , `encounters:amend` are all `SECURITY DEFINER` RPCs that re-check caller identity inside the function. No direct table mutations from the client.
6. **Audit-write side effect** — actions marked ★ must insert into `audit.audit_log` as part of the same transaction. If the audit insert fails, the action rolls back.

---

8. Edge cases

1. **Multi-tenant users** — a user who belongs to two practices has two `tenant_members` rows. The active tenant is resolved by subdomain (`TENANT_ROUTING_MODE=subdomain`); `getSession()` picks the matching membership. Cross-tenant tasks/appointments are **not allowed**.
2. **User with zero active memberships** — soft-deleted from every practice. Treated as unauthenticated (redirected to `/login`). Do not show "invited but not joined" UI yet; that comes with the invitation flow.
3. **Platform admin with no impersonation** — can hit `/admin` on the platform surface (cross-tenant dashboards) but NOT any tenant-scoped route. Must start an impersonation session to access a tenant.
4. **Last owner protection** — `tenant_members_enforce_owner` trigger (already live) blocks removing the final `practice_owner` . Offboarding UI must detect this and force role-transfer first.
5. **Patient user exists in staff context** — if a staff member has a patient record at the same practice (e.g., the physician's own chart), their patient data is visible only via `self:*` permissions in the patient portal, never via staff routes.
6. **Clock skew on impersonation expiry** — session expiry is DB-side (`now() > expires_at`). Don't check in the browser.
7. **Deletion vs. amendment** — amending a signed note does **not** replace it; the original stays, amended version is a new row linked via `amended_from_id` . The "delete note" action is disabled for signed notes.
8. **Role grant during impersonation** — `admin:users` is stripped by impersonation (rbac.ts:100-106). A super_admin impersonating a practice_owner cannot hand out new roles.
9. **Bulk export as a permission** — `patient_records:export` , `notes:export` , `audit_logs:export` , `claims:export` are *distinct* from `view` because they generate PHI artifacts that leave the app boundary. Always audit-logged.
10. **Service users** (integrations) — `user_kind='service'` with a limited permission set (usually `patient:read` , `billing:read` + integration-specific writes). Bound to an API key, no login UI.
11. **Feature-gated modules** — a tenant on `plan='starter'` may not have `claims` enabled. Check `entitlements` before showing the nav item; RLS on `claims` table should also enforce.

---

9. Security risks & mitigations

Risk	Mitigation
Developer forgets a permission check in a route	RLS catches it at the DB layer
TS permission map drifts from SQL	CI test diffs them (TODO — not yet written)
Impersonation abused for edits	Sensitive perms stripped + all actions audit-logged with `impersonator_id`
Patient record export leaked	`*:export` perms separate from `view` ; all exports audit-logged; UI confirms intent
Tenant context forgotten in a query	`SECURITY INVOKER` + RLS; no `SECURITY DEFINER` without explicit tenant param
Privilege escalation via role self-edit	`staff_records:update` excludes role changes; role changes require `staff_records:manage`
Stale session after offboarding	Middleware refreshes session each request; `tenant_members.deleted_at` causes `getSession()` to return null on next tick
Patient portal user gains staff access	Separate `user_kind` , separate surface, no shared routes; `permissionsFor()` returns empty for non-staff kinds
Cross-practice data bleed	All tables `tenant_id NOT NULL` ; RLS predicates uniformly apply `is_member_of(tenant_id)`
Weak passwords	Supabase min length 12 + HIBP check (enable in Dashboard); MFA required before v1.0

---

10. Migration from current coarse permissions

Current rbac.ts has ~30 coarse keys (`clinical:write` , `billing:read`). The V1 matrix has ~90 keys.

Strategy: two-phase, non-breaking.

Phase 1 (1 PR): Add the new `permissions-v2.ts` alongside the existing `rbac.ts` . Introduce `hasPermissionV2()` helpers. Start annotating new routes/actions with V2 keys. Existing routes keep using V1 keys.

Phase 2 (subsequent PRs, one surface at a time):

- Translate V1 → V2 for one surface.
- Update the matching RLS policies in a new migration (additive: `has_permission_v2()` SQL function).
- Remove V1 keys for that surface once all call sites converted.
- Drop V1 keys + `has_permission()` after last surface migrated.

Drift check (add in Phase 1):

```
// packages/auth/test/permission-sql-sync.test.ts
test("TS ROLE_PERMISSIONS matches public.has_permission()", async () => {
  const sqlRows = await sql`SELECT role, permission FROM role_permissions_view`;
  expect(roleMapAsRows(ROLE_PERMISSIONS_V2)).toEqual(sqlRows);
});
```

---

11. Recommended TypeScript API

See `packages/auth/src/permissions-v2.ts` for the generated constants, role-map, and helpers.

Key exports:

- `MODULES` , `ACTIONS` — const arrays (source of truth for enum generation).
- `PERMISSIONS_V2` — frozen array of all valid `module:action` keys.
- `ROLE_PERMISSIONS_V2` — role → Permission[] map.
- `hasPermissionV2(ctx, perm)` / `requirePermissionV2(ctx, perm)` — guards.
- `permissionsForRolesV2(roles, opts)` — same signature as today's `permissionsFor` , returns V2 set.
- `filterModuleActions(roles, module)` — returns the allowed actions for a given module (useful for UI gating).